# Proposal: SKL as Single Source of Truth, Publishing into Visier

Oct 29, 2026 · @jeffrey

## Summary

We propose the SKL as the single source of truth (SSOT) for vocabularies, taxonomies, ontologies, rules, constraints, logic and calculations, with governed outbound services that publish the parts Visier needs. Visier becomes a consumption-only execution point: nothing is authored or edited there.

**Decision requested:** approve (1) the SKL-as-SSOT principle, (2) API-first publishing as the standard pattern for downstream execution points, and (3) a pilot scoped to the downturn workforce advisory vocabularies, with Visier as the first target.

## Context

Teams are curating semantics inside Visier, which is an analytics semantic layer, not a knowledge-management system. Its public documentation shows the limits:

- Visier models subjects, properties, dimensions, concepts and metrics. Its Dimensions API is Alpha and limited to leveled and parent-child dimensions.
- Its SQL-like query language works within one tenant, and queries across analytic objects use only the attributes those objects share. I found no public documentation of RDF/OWL import, SPARQL, reasoning or federated queries.
- Its skills ontology (14,000+ skills, 3,300+ occupations, built on ESCO) is a product feature, and customizing it is listed as a future capability.

If Visier becomes the master, poly-hierarchy, typed relationships, mappings and rules have nowhere to live, and the other consumers (Copilot, Compass, HRScale, partner APIs) cannot reuse what is curated there. Two masters will drift.

## Proposal

The SKL is the only place meaning is authored. Three layers, with a one-way flow from authority to execution:

1. **SKL (authority):** authoring, governance, versioning and the canonical store for vocabularies, taxonomies, ontologies, mappings, rules, constraints, logic and calculations.
2. **Outbound publishing services (API layer):** project SKL content into the shape a target needs, validate it, deliver it, and reconcile what the target holds. One service contract, one adapter per execution point.
3. **Execution points:** Visier first, others later on the same contract. They consume published content and never edit it.

Visier reports back only reconciliation status (what it holds, and whether it matches). It never sends definitions back.

## Fit with architecture principles

The first four principles come from the Keel & North Star deck; the rest are first-principles additions. Confirm the list against enterprise standards before circulating.

| Principle | How the proposal satisfies it |
|---|---|
| Single source of truth for rules, vocabularies, taxonomies, ontologies | The SKL is the only place meaning is authored. Visier holds projections. |
| Extend platforms, don't replace them | Visier stays the analytics engine. The SKL adds the definitions it cannot hold. |
| Deterministic core | Eligibility, compliance and calculations run as governed logic in services. Visier receives results and dimensions and does not reimplement the rules. |
| Reusable services assembled per tier | Publishing is one service contract with an adapter per target. |
| Define once, publish many | One definition feeds Visier, Copilot, Compass, HRScale and partner APIs. |
| Separate meaning from execution | Definitions live in the SKL. Execution points apply them. |
| Consumption only downstream | Published objects are locked in Visier. Changes start in the SKL. |

| Principle | How the proposal satisfies it |
|---|---|
| Traceable and auditable | Every published object carries its SKL ID and version, and every publish is recorded. |
| Vendor-neutral and reversible | A different analytics tool means a new adapter and a republish, not a re-curation. |

## Benefits

- **One definition everywhere:** a term, hierarchy or calculation means the same thing in Visier, Copilot, Compass and partner APIs.
- **Richer semantics than Visier can hold:** poly-hierarchy, typed relationships, mappings and deontic norms stay intact in the SKL.
- **Lower cost to serve:** one governed change replaces parallel edits in each tool, and fewer reconciliation tickets.
- **Audit trail:** every recommendation and metric traces to a versioned SKL definition, which supports MTBH and defensibility.
- **Safer change:** versioned, reviewed and tested publishes, with drift detected instead of discovered.
- **Customer-specific terminology:** customer vocabularies map to a canonical set once and publish per tenant.
- **Less lock-in:** Visier is replaceable or supplementable without redoing the semantics.

## What is authored, what is published

The SKL authors everything. Only the parts that fit a Visier object type are published; the rest stays in the SKL.

| Element | Authored in the SKL as | Published into Visier as | Notes |
|---|---|---|---|
| Vocabularies (terms, definitions, synonyms) | Concepts with ID, labels, definition, status | Dimension members or property values, each with its SKL ID | Synonyms and definitions stay in the SKL |

| Element | Authored in the SKL as | Published into Visier as | Notes |
|---|---|---|---|
| Taxonomies (hierarchies) | Concept schemes with broader/narrower relations | Leveled or parent-child dimensions (one parent path per member) | Multiple parents handled by the rule in the next section |
| Ontology (classes, relations, axioms) | Classes, typed relations, constraints | Not published; flattened projections only | Stays in the SKL |
| Mappings and crosswalks | Match relations between vocabularies (exact, close, broad, narrow) | Mapping tables used at data load | Authoritative copy stays in the SKL |
| Selection criteria | Declarative rule definitions | Selection concepts and calculated properties, where expressible | What Visier cannot express stays in services |
| Rules, constraints, deontic norms | Governed logic with test cases | Not published; executed in deterministic services, with results loadable | Stays in the SKL |
| Calculations and metrics | Definition with test cases | Metric formula as an implementation of the SKL definition | Parity-tested; the SKL definition wins |
| Skills | ESCO-aligned reference set plus customer terms | To be decided | See the skills section |
| Provenance | SKL ID, version, publish record | A property on every published object | Enables drift detection |

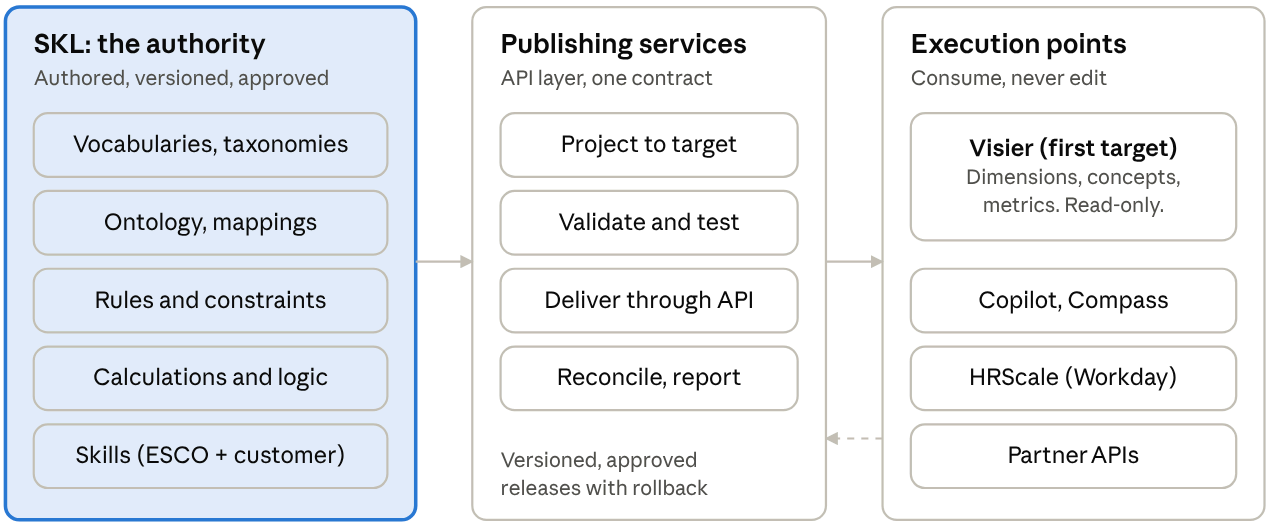
## Operational decisions

1. **Consumption only downstream.** Published dimensions, concepts and metrics in Visier are locked. Any change starts in the SKL and flows through a publish. A reconciliation job flags any object in Visier that differs from its SKL source.
2. **What stays in the SKL.** Ontology axioms and relations, mappings, rules, constraints, deontic norms, MTBH logic, and eligibility and compliance calculations. Services execute the logic. Visier receives results and dimensions.
3. **Concepts with multiple parents use a polymorphic relation.** The canonical concept stays single in the SKL. For each additional parent context, the SKL mints a new fully qualified term (for example, Parent A > Term) with its own ID, a reference to that parent, and a link to the canonical concept ID. Visier receives one parent path per term, and the canonical ID is carried as a property so analysis can still roll up by the single concept. Trade-off: summing across contexts double-counts, so metrics must count by canonical ID or use one chosen view.
4. **Publishing goes through the API layer.** The publishing service calls Visier's APIs, batching changes into a project (using the ProjectID header) so a publish lands as one production version instead of hundreds. Fallback: file-based or Direct Data Intake loading. Gate: confirm the status of the Dimensions API (currently Alpha) and its dimension-type limits with Visier before committing to it.
5. **Versioned, reviewed releases.** Each publish has a version, release notes, an approval, and a rollback path. Nothing reaches production Visier without passing validation and the parity checks.

## Outbound publishing services

The publishing layer is a service with one contract and one adapter per execution point, so adding a target does not change the SKL.

**The SKL authors once; Visier only consumes what is published**




Solid arrows: publish. Dashed arrow: reconciliation status only, never definitions.

SKL, publishing services and execution points · one-way flow

Publishing runs one way, with reconciliation status coming back so drift is seen early.

- **Publish:** projects a versioned SKL release into the target's shape and delivers it.
- **Validate:** checks structure (for example, the dimension types the target supports), unique IDs and parity tests before anything is delivered.
- **Reconcile:** reads back what the target holds and compares it to the release.
- **Status and rollback:** reports release state and reverts to the previous version.
- **Visier adapter:** maps concept schemes to Visier dimension types and selection concepts, writes through Visier's APIs inside a project, and stamps the SKL ID and version on every published object.

## Skills (to be decided)

Proposed direction: host the ESCO-aligned skills ontology in the SKL as the canonical reference, treat each customer's own skills and terminology as local terms mapped to it, and publish the result downstream per tenant. This status is TBD pending the open questions below.

| Option | What it means | Trade-off |
|---|---|---|
| A. Map ESCO in the SKL, expose downstream (proposed) | The SKL holds the canonical skills and customer terms, linked by match relations (exact, close, broad, narrow). Publishing writes the mapped result into Visier. | Most control and reuse; requires reconciling IDs with Visier's own skills ontology. |
| B. Adopt Visier's skills ontology as is | Use the shipped 14,000+ skills and their hierarchy. | Fastest; customer terms and customization are limited, and customization is listed as future. |
| C. Per-customer skill sets only | Each customer's skills load as their own dimension. | Fits each customer; no shared canonical set, so benchmarking and reuse suffer. |

Open questions:

- How do Visier's skill IDs relate to ESCO identifiers, and how much has Visier changed from ESCO?
- Do we load alongside Visier's skills dimension or replace it, and what does that do to its prebuilt metrics?
- Which ESCO version and licence terms apply, and who owns updates?
- Who approves a customer term that maps to nothing in the canonical set?

## Target operating model

The model follows from seven first principles. Everything else (roles, process, measures) is derived from them.

1. **Meaning has one owner.** Every term, rule and calculation has one accountable steward and one home, the SKL.
2. **Define once, publish many.** Targets are projections, never sources.
3. **Separate definition from execution.** The SKL defines; services and platforms execute.

4. **Downstream consumes, never edits.** Anything changed downstream is drift, and drift is a defect.
5. **Every published fact is traceable.** It carries its SKL ID, version and publish record.
6. **Automate the mechanical, govern the judgemental.** Projection, validation and reconciliation are automated. Naming, scoping and approving meaning are human decisions.
7. **Measure the system.** If drift, lead time and cost to serve are not measured, the model will not hold.

## Roles

| Role | Accountable for |
|---|---|
| Semantic architect | SKL structure, modeling standards, the publishing contract and the multi-parent rule |
| Domain stewards (HR, benefits, compliance) | Meaning and correctness of terms, rules and calculations in their domain |
| Semantic engineers | The SKL platform, publishing services and per-target adapters |
| Visier model developer | Applying published releases in Visier, and blocking local edits to published objects |
| Governance council | Approving releases, resolving cross-domain conflicts and customer-term requests |

## Lifecycle of a change

1. **Propose:** a steward raises a change with its business reason.
2. **Author:** the change is modeled in the SKL with definitions and test cases.
3. **Review:** the steward and architect check meaning and fit.
4. **Validate:** automated checks run on structure, IDs and parity tests.
5. **Approve and release:** the council approves a versioned release.
6. **Publish:** the service delivers it to Visier through the API layer, inside a project.
7. **Reconcile:** the service reads back Visier and flags any difference from the release.
8. **Learn:** drift and incidents feed back into standards and tests.

### Measures

- Share of Visier semantic objects sourced from the SKL
- Drift incidents per release, and time to resolve
- Parity-test failures caught before publish
- Lead time from approved change to live in Visier
- Cost to serve for the downturn advisory pilot, as in the baseline the deck proposes

## Risks, pilot and next steps

**Pilot scope:** the downturn workforce advisory vocabularies (for example, employment actions such as furlough, reduced hours and reduction in force, with their reasons and benefit classes). Author them in the SKL, publish a small set of dimensions and selection concepts into Visier, and run the reconciliation and parity checks end to end. Baseline cost to serve first.

| Risk | Mitigation |
|---|---|
| Dimensions API is Alpha and limited to leveled and parent-child types | Confirm status and roadmap with Visier; keep file-based or Direct Data Intake loading as the fallback |
| Visier admins edit published objects | Lock them, and reconcile on every release |
| Flattening loses semantics (multiple parents, relations) | Fully qualified terms with canonical IDs; keep ontology in the SKL |
| Summing across qualified terms double-counts | Metrics count by canonical ID, or use a single chosen view |
| Calculation drift between the SKL and Visier metrics | Test cases in the SKL, with parity tests on every release |
| Skills IDs do not line up with ESCO or Visier's ontology | Resolve in the skills decision before any skills publish |

Next steps:

- [ ] Confirm with Visier: Dimensions API status, dimension-type limits, project-based publishing, SQL-like join and reference support
- [ ] Agree the multi-parent naming and ID convention

- [ ] Define the publishing service contract and the Visier adapter
- [ ] Select the pilot vocabularies and name their stewards
- [ ] Settle the skills option and the open questions above
- [ ] Stand up the governance council and release process

**Sources:** Visier documentation, reviewed through search results and worth verifying against your tenant: Understand Visier's Analytic Model, Dimensions API, SQL-like queries, Skills Insights. Internal: Navigating Rough Seas (Keel & North Star) deck.